

my_TB.sv > Verilog-HDL/SystemVerilog/Bluespec SystemVerilog > { } my_TB

```
import param_pkg::*;
import axi4_class::*;

module my_TB (
    axi4_if.master master
);

    axi4_pkt object;
    // Associative array that mimics the memory of slave
    logic [DATA_SIZE-1:0] golden_arr[logic [ADDR_SIZE-1:0]];

    ////////////////////////////////////////////////// internal signal to flag phase start and end //////////////////////////////////////////////////
    ////////////////////////////////////////////////// internal signal to flag phase start and end //////////////////////////////////////////////////
    ////////////////////////////////////////////////// internal signal to flag phase start and end //////////////////////////////////////////////////

    typedef enum logic {
        write,
        read
    } op_e;

    op_e operation = write;
    logic flag = 0;

    ////////////////////////////////////////////////// TASKS DEFINITIONS //////////////////////////////////////////////////
    ////////////////////////////////////////////////// TASKS DEFINITIONS //////////////////////////////////////////////////
    ////////////////////////////////////////////////// TASKS DEFINITIONS //////////////////////////////////////////////////
```

```
module my_TB (

    ////////////////////////////////////////////////// TASKS DEFINITIONS //////////////////////////////////////////////////

    task automatic generate_stim(input axi4_pkt obj);
        assert (obj.randomize())
        else $fatal("Randomization gone wrong");
    endtask

    task automatic update_object(input axi4_pkt obj);
        obj.AWVALID = master.AWVALID;
        obj.AWREADY = master.AWREADY;
        obj.WVALID = master.WVALID;
        obj.WREADY = master.WREADY;
        obj.ARVALID = master.ARVALID;
        obj.ARREADY = master.ARREADY;
        obj.RVALID = master.RVALID;
        obj.RREADY = master.RREADY;
        obj.BVALID = master.BVALID;
        obj.BRESP = axi_resp_e'(master.BRESP);
        obj.BREADY = master.BREADY;
        obj.WLAST = master.WLAST;
        obj.RLAST = master.RLAST;
        obj.RRESP = axi_resp_e'(master.RRESP);
        obj.sampled_rdata = master.RDATA;
        obj.sampled_wdata = master.WDATA;
    endtask

    task automatic drive_addr_write(input axi4_pkt obj);
        // master signal
    endtask
```

```

6
7 task automatic drive_addr_write(input axi4_pkt obj);
8
9     // master signals
10    master.AWADDR = obj.AWADDR;
11    master.AWLEN  = obj.AWLEN;
12    master.AWSIZE = obj.AWSIZE;
13
14 endtask
15
16 task automatic drive_addr_read(input axi4_pkt obj);
17     if (!flag) begin
18         master.ARADDR = obj.ARADDR;
19         master.ARLLEN = obj.ARLLEN;
20         master.ARSIZE = obj.ARSIZE;
21     end else begin
22         master.ARADDR = obj.AWADDR;
23         master.ARLLEN = obj.AWLEN;
24         master.ARSIZE = obj.AWSIZE;
25         obj.R_beats  = obj.W_beats;
26     end
27
28 endtask
29
30 task automatic drive_data(input axi4_pkt obj);
31     master.WVALID = 1;
32     @(negedge master.ACLK);
33     for (int i = 0; i < obj.W_beats; i++) begin
34         master.WDATA = obj.WDATA[i];
35         if (i == (obj.WDATA.size() - 1)) master.WLAST = 1;

```

```

module my_TB (
    task automatic drive_data(input axi4_pkt obj);

        master.WDATA = obj.WDATA[i];
        if (i == (obj.WDATA.size() - 1)) master.WLAST = 1;
        wait (master.WREADY);
        update_object(obj);
        obj.cg.sample();
        @(negedge master.ACLK);
    end
    master.WVALID = 0;
    master.WLAST = 0;
endtask

task automatic golden_model(input axi4_pkt obj);
    int unsigned address = (obj.AWADDR >> 2); // MUST be unsigned

    for (int i = 0; i < obj.W_beats; i++) begin
        if (address < 1023) begin
            golden_arr[address] = obj.WDATA[i];
        end else begin
            $display("Golden Model: Write rejected by DUT at addr %h", address);
        end
        address++;
    end
endtask

//////////////////// CHECK DATA //////////////////////

```

```

110 ////////////////////////////////////////////////// CHECK DATA //////////////////////////////////////
111
112 task automatic check_data(input axi4_pkt obj);
113     int address;
114     address = (iflag) ? int'(master.ARADDR >> 2) : int'(master.AWADDR >> 2);
115
116     master.RREADY = 1;
117     for (int i = 0; i < obj.R_beats; i++) begin
118         @(posedge master.ACLK);
119         while (!master.RVALID) @(posedge master.ACLK);
120
121         if (master.RRESP == 2'b10) begin // SLVERR
122             if (master.RDATA == 32'h0000_0000) begin
123                 $display("ID: %0d, DUT safely blocked illegal read at address %h (SLVERR).", i, address);
124             end else begin
125                 $error("ID: %0d, DUT threw SLVERR but leaked data %h at address %h", i, master.RDATA,
126                     address);
127             end
128         end
129         else begin
130             if (golden_arr.exists(address)) begin
131                 if (golden_arr[address] == master.RDATA) begin
132                     $display("ID: %d , Correct READ: Expected data: %h, at address: %h Actual Data: %h", i,
133                         golden_arr[address], address, master.RDATA);
134                 end else begin
135                     $error("ID: %d Expected data: %h at address: %h , Actual Data: %h", i,
136                         golden_arr[address], address, master.RDATA);
137                 end
138             end else begin
139                 $display("This memory location is empty");
140             end
141         end
142     end
143 endtask

```

```

12 task automatic check_data(input axi4_pkt obj);
13     end else begin
14         $display("This memory location is empty");
15     end
16 end
17
18 address++;
19 update_object(obj);
20 obj.cg.sample();
21 if (i == obj.R_beats - 1) obj.cg.sample();
22 end
23 master.RREADY = 0;
24 endtask
25
26 task automatic write_op(input axi4_pkt obj, ref op_e op, ref logic flag);
27
28     if (op == write) begin
29         @(negedge master.ACLK);
30         generate_stim(obj);
31
32         // WRITE ADDRESS PHASE
33
34         master.AWVALID = 1;
35         drive_addr_write(obj);
36         wait (master.AWREADY);
37         update_object(obj);
38         obj.cg.sample();
39         @(negedge master.ACLK);

```

```

40 task automatic write_op(input axi4_pkt obj, ref op_e op, ref logic flag);
41
42     @(negedge master.ACLK);
43     master.AWVALID = 0;
44
45     // DATA PHASE
46     drive_data(obj);
47     golden_model(obj);
48
49     // RESPONSE PHASE
50     master.BREADY = 1;
51     wait (master.BVALID);
52     update_object(obj);
53     obj.cg.sample();
54     @(negedge master.ACLK);
55     master.BREADY = 0;
56     flag = 1;
57     op = read;
58 end
59
60 endtask

```

```

task automatic read_op(input axi4_pkt obj, ref op_e op, ref logic flag);

    if (op == read) begin
        if (!flag) generate_stim(obj);

        // READ ADDRESS PHASE
        master.ARVALID = 1;
        drive_addr_read(obj);
        wait (master.ARREADY);
        update_object(obj);
        obj.cg.sample();
        @(negedge master.ACLK);
        master.ARVALID = 0;

        // READ DATA PHASE
        check_data(obj);
        master.RREADY = 0;

        //WRITE BACK READ
        op = write;
        flag = 0;
    end
endtask

```

```

task automatic protocol(input axi4_pkt obj, int size, ref op_e op, ref logic flag);

    for (int i = 0; i < size; i++) begin
        $display("Transaction no.: %0d, operation: %s", i, op.name());
        write_op(obj, op, flag);
        read_op(obj, op, flag);
    end
endtask

initial begin
    master.AWVALID = 0;
    master.WVALID = 0;
    master.ARVALID = 0;
    master.RREADY = 0;
    master.BREADY = 0;
    master.WLAST = 0;
    master.WDATA = 0;
    master.AWADDR = 0;
    master.ARADDR = 0;
    object = new();
    wait (master.ARESETn);
    @(negedge master.ACLK);
    protocol(object, 2000, operation, flag);
    $stop;
end
endmodule

```

AXI class:

```
package axi4_class;

import param_pkg::*;

class axi4_pkt;

    //////////////////////////////////////
    // 1. WRITE SIGNALS
    //////////////////////////////////////

    rand logic [ADDR_SIZE-1:0] AWADDR;
    rand logic [7:0] AWLEN;
    rand logic [2:0] AWSIZE;
    logic AWVALID;
    logic AWREADY;

    rand logic [DATA_SIZE-1:0] WDATA [];
    logic WVALID;
    logic WLAST;
    logic WREADY;

    axi_resp_e BRESP;
    logic BVALID;
    logic BREADY;

    //////////////////////////////////////
    // 2. READ SIGNALS
    //////////////////////////////////////

    rand logic [ADDR_SIZE-1:0] ARADDR;
    rand logic [7:0] ARLEN;
    rand logic [2:0] ARSIZE;
    logic ARVALID;
    logic ARREADY;

endclass
```

```

logic                                ARVALID;
logic                                ARREADY;

logic                                [DATA_SIZE-1:0] RDATA        [];
axi_resp_e                           RRESP;
logic                                RVALID;
logic                                RLAST;
logic                                RREADY;

////////////////////////////////////
// 3. INTERNAL VARIABLES & HELPERS
////////////////////////////////////

int                                    W_beats;
int                                    R_beats;

logic                                [DATA_SIZE-1:0] sampled_wdata;
logic                                [DATA_SIZE-1:0] sampled_rdata;
int                                    last_addr;

function void post_randomize();
    this.W_beats = AWLEN + 1;
    this.R_beats = ARLEN + 1;
    this.RDATA    = new[ARLEN + 1];
endfunction

```

```

////////////////////////////////////
// 4. WRITE CONSTRAINTS
////////////////////////////////////

constraint wsize_c {AWSIZE == 3'd2;}

constraint awaddr_c {
    AWADDR dist {
        16'd0           := 5, // 5% chance of hitting min address
        16'd4092        := 5, // 5% chance of hitting max valid word address
        [16'd1 : 16'd4091] :/ 90 // 90% normal randomization
    };
}

constraint wlen_c {
    AWLEN dist {
        8'd0             := 5, // 5% chance of 1 beat
        8'd255           := 5, // 5% chance of 256 beats
        [8'd1 : 8'd254] :/ 90
    };
}

constraint wdata_c {
    WDATA.size() == AWLEN + 1;
    foreach (WDATA[i]) {WDATA[i] inside {[32'h0000_0000 : 32'hFFFF_FFFF]};}
}

```

```

class axi4_pkt;

    constraint wdata_c {
        WDATA.size() == AWLEN + 1;
        foreach (WDATA[i]) {WDATA[i] inside {[32'h0000_0000 : 32'hFFFF_FFFF]};}
    }

    //////////////////////////////////////////////////
    // 5. READ CONSTRAINTS
    //////////////////////////////////////////////////
    constraint arsize_c {ARSIZE == 3'd2;}

    constraint araddr_c {
        ARADDR dist {
            16'd0           := 5,
            16'd4092        := 5,
            [16'd1 : 16'd4091] :/ 90
        };
    }

    constraint arlen_c {
        ARLEN dist {
            8'd0           := 5,
            8'd255         := 5,
            [8'd1 : 8'd254] :/ 90
        };
    }

```

```

class axi4_pkt;

```

```

    //////////////////////////////////////////////////
    // 6. COVERGROUP
    //////////////////////////////////////////////////
    covergroup cg;
        option.auto_bin_max = 0; // Remove automatic bins globally
        option.per_instance = 1; // Highly recommended for class-based covergroups

        // =====
        // WRITE PHASE COVERAGE
        // =====

        // -- Write Address Handshake --
        master_AWvalid: coverpoint AWVALID {
            bins one = {1'b1}; bins zero = {1'b0};
        }
        slave_AWready: coverpoint AWREADY {bins one = {1'b1}; bins zero = {1'b0};}

        W_addr_handshake: cross master_AWvalid, slave_AWready{
            option.cross_auto_bin_max = 0;
            bins addr = binsof (master_AWvalid.one) && binsof (slave_AWready.one);
            ignore_bins ignore_zeros = binsof (master_AWvalid.zero) || binsof (slave_AWready.zero);
        }

        // -- Write Data Handshake --
        master_Wvalid: coverpoint WVALID {
            bins one = {1'b1}; bins zero = {1'b0};
        }
        slave_Wready: coverpoint WREADY {bins one = {1'b1}; bins zero = {1'b0};}
        W_last: coverpoint WLAST {bins one = {1'b1}; bins zero = {1'b0};}

```



```

write_handshake: cross master_Wvalid, slave_Wready, W_last{
    option.cross_auto_bin_max = 0;
    bins write_start = binsof(slave_Wready.one) && binsof(master_Wvalid.one) && binsof(W_last.zero);
    bins write_end   = binsof(slave_Wready.one) && binsof(master_Wvalid.one) && binsof(W_last.one);
    ignore_bins ignore_zeros = binsof(master_Wvalid.zero) || binsof(slave_Wready.zero);
}

// -- Write Response Handshake --
Bvalid_cv: coverpoint BVALID {
    bins one = {1'b1}; bins zero = {1'b0};
}
Bready_cv: coverpoint BREADY {bins one = {1'b1}; bins zero = {1'b0};}

Bresp_check: cross Bready_cv, Bvalid_cv{
    option.cross_auto_bin_max = 0;
    bins resp_done = binsof(Bready_cv.one) && binsof(Bvalid_cv.one);
    ignore_bins ignore_zeros = binsof(Bready_cv.zero) || binsof(Bvalid_cv.zero);
}

// -- Write Protocol Vectors --
W_address_regions: coverpoint AWADDR {
    bins w_addr_min = {16'd0};
    bins w_addr_low = {[16'd1 : 16'd1000]};
    bins w_addr_mid = {[16'd1001 : 16'd3000]};
    bins w_addr_high = {[16'd3001 : 16'd4091]};
    bins w_addr_max = {16'd4092};
}

```

```

W_burst_length: coverpoint AWSIZE {
    bins w_size = {3'd2}; illegal_bins illegal_w = {[3'd0 : 3'd1], [3'd3 : 3'd7]};
}

w_len: coverpoint AWLEN {
    bins w_len_min = {8'd0};
    bins w_len_low = {[8'd1 : 8'd80]};
    bins w_len_mid = {[8'd81 : 8'd160]};
    bins w_len_high = {[8'd161 : 8'd254]};
    bins w_len_max = {8'd255};
}

w_resp: coverpoint BRESP {bins okay = {OKAY}; bins slverr = {SLVERR};}

write_boundary: cross w_len, W_address_regions, w_resp{
    option.cross_auto_bin_max = 0;
    bins max   = binsof(w_len.w_len_max) && binsof(W_address_regions.w_addr_max) && binsof(w_resp.slverr);
    bins max_ = binsof(w_len.w_len_max) && binsof(W_address_regions.w_addr_min) && binsof(w_resp.okay);
    bins mid_ = binsof(w_len.w_len_mid) && binsof(W_address_regions.w_addr_min) && binsof(w_resp.okay);
    bins min_ = binsof(w_len.w_len_min) && binsof(W_address_regions.w_addr_min) && binsof(w_resp.okay);
}

```

```

// -----
// READ PHASE COVERAGE
// =====

// -- Read Address Handshake --
master_ARvalid: coverpoint ARVALID {
  bins one = {1'b1}; bins zero = {1'b0};
}
slave_ARready: coverpoint ARREADY {bins one = {1'b1}; bins zero = {1'b0};}

R_addr_handshake: cross master_ARvalid, slave_ARready{
  option.cross_auto_bin_max = 0;
  bins addr_done = binsof (master_ARvalid.one) && binsof (slave_ARready.one);
  ignore_bins ignore_zeros = binsof (master_ARvalid.zero) || binsof (slave_ARready.zero);
}

// -- Read Data Handshake --
master_Rready: coverpoint RREADY {
  bins one = {1'b1}; bins zero = {1'b0};
}
slave_Rvalid: coverpoint RVALID {bins one = {1'b1}; bins zero = {1'b0};}
R_last: coverpoint RLAST {bins one = {1'b1}; bins zero = {1'b0};}

read_handshake: cross slave_Rvalid, master_Rready, R_last{
  option.cross_auto_bin_max = 0;
  bins read_start = binsof(master_Rready.one) && binsof(slave_Rvalid.one) && binsof(R_last.zero);
  bins read_end   = binsof(master_Rready.one) && binsof(slave_Rvalid.one) && binsof(R_last.one);
  ignore_bins ignore_zeros = binsof (master_Rready.zero) || binsof (slave_Rvalid.zero);
}

```

```

class axi4_pkt;
// -- Read Protocol Vectors --
R_address_regions: coverpoint ARADDR {
  bins r_addr_min = {16'd0};
  bins r_addr_low = {[16'd1 : 16'd1000]};
  bins r_addr_mid = {[16'd1001 : 16'd3000]};
  bins r_addr_high = {[16'd3001 : 16'd4091]};
  bins r_addr_max = {16'd4092};
}

R_burst_length: coverpoint ARSIZE {
  bins r_size = {3'd2}; illegal_bins illegal_r = {[3'd0 : 3'd1], [3'd3 : 3'd7]};
}

r_len: coverpoint ARLEN {
  bins r_len_min = {8'd0};
  bins r_len_low = {[8'd1 : 8'd80]};
  bins r_len_mid = {[8'd81 : 8'd160]};
  bins r_len_high = {[8'd161 : 8'd254]};
  bins r_len_max = {8'd255};
}

r_resp: coverpoint RRESP {bins okay = {OKAY}; bins slverr = {SLVERR};}

read_boundary: cross r_len, R_address_regions, r_resp{
  option.cross_auto_bin_max = 0;
  bins max   = binsof(r_len.r_len_max) && binsof(R_address_regions.r_addr_max) && binsof(r_resp.slverr);
  bins max_ = binsof(r_len.r_len_max) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
  bins mid_ = binsof(r_len.r_len_mid) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
  bins min_ = binsof(r_len.r_len_min) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
}

```

```

class axi4_pkt;
// -- Read Protocol Vectors --
R_address_regions: coverpoint ARADDR {
    bins r_addr_min = {16'd0};
    bins r_addr_low = {[16'd1 : 16'd1000]};
    bins r_addr_mid = {[16'd1001 : 16'd3000]};
    bins r_addr_high = {[16'd3001 : 16'd4091]};
    bins r_addr_max = {16'd4092};
}

R_burst_length: coverpoint ARSIZE {
    bins r_size = {3'd2}; illegal_bins illegal_r = {[3'd0 : 3'd1], [3'd3 : 3'd7]};
}

r_len: coverpoint ARLEN {
    bins r_len_min = {8'd0};
    bins r_len_low = {[8'd1 : 8'd80]};
    bins r_len_mid = {[8'd81 : 8'd160]};
    bins r_len_high = {[8'd161 : 8'd254]};
    bins r_len_max = {8'd255};
}

r_resp: coverpoint RRESP {bins okay = {OKAY}; bins slverr = {SLVERR};}

read_boundary: cross r_len, R_address_regions, r_resp{
    option.cross_auto_bin_max = 0;
    bins max_ = binsof(r_len.r_len_max) && binsof(R_address_regions.r_addr_max) && binsof(r_resp.slverr);
    bins max_ = binsof(r_len.r_len_max) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
    bins mid_ = binsof(r_len.r_len_mid) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
    bins min_ = binsof(r_len.r_len_min) && binsof(R_address_regions.r_addr_min) && binsof(r_resp.okay);
}

```

```
endgroup
```

```
function new();
```

```
    cg = new();
```

```
endfunction
```

```
endclass
```

```
endpackage
```

Interface + assertions:

```
axi4_interface.sv > Verilog-HDL/SystemVerilog/Bluespec SystemVerilog > axi4_if
1  import param_pkg::*;
2
3  interface axi4_if #(
4      parameter ADDR_WIDTH      = ADDR_SIZE,
5      parameter DATA_WIDTH     = DATA_SIZE,
6      parameter DEPTH           = DEPTH_SIZE,
7      parameter MEM_ADDR_WIDTH = MEM_ADDR_SIZE
8  ) (
9      input ACLK,
10     input bit ARESETn
11 );
12
13
14     ////////// WRITE signals //////////
15
16     logic [ADDR_WIDTH-1:0] AWADDR;
17     logic [7:0] AWLEN;
18     logic [2:0] AWSIZE;
19     logic AWVALID;
20     logic AWREADY;
21     logic [DATA_WIDTH-1:0] WDATA;
22     logic WVALID;
23     logic WLAST;
24     logic WREADY;
25     logic [1:0] BRESP;
26     logic BVALID;
27     logic BREADY;
28
29     ////////// READ signals //////////
30
```

```
////////// READ signals //////////
```

```
logic [ADDR_WIDTH-1:0] ARADDR;  
logic [7:0] ARLEN;  
logic [2:0] ARSIZE;  
logic ARVALID;  
logic ARREADY;  
logic [DATA_WIDTH-1:0] RDATA;  
logic [1:0] RRESP;  
logic RVALID;  
logic RLAST;  
logic RREADY;
```

```
////////// MEMORY signals //////////
```

```
logic mem_en;  
logic mem_we;  
logic [MEM_ADDR_WIDTH-1:0] mem_addr;  
logic [DATA_WIDTH-1:0] mem_wdata;  
logic [DATA_WIDTH-1:0] mem_rdata;
```

```
modport slave(  
    input ACLK, ARESETn, AWADDR, AWLEN, AWSIZE, AWVALID, WDATA, WVALID, WLAST,  
           BREADY, ARADDR, ARLEN, ARSIZE, ARVALID, RREADY,  
    output AWREADY, WREADY, BRESP, BVALID, ARREADY, RDATA, RRESP, RVALID, RLAST  
);  
modport master(  
    output AWADDR, AWLEN, AWSIZE, AWVALID, WDATA, WVALID, WLAST, BREADY, ARADDR, ARLEN, ARSIZE, ARVALID, RREADY,  
    input ACLK, ARESETn, AWREADY, WREADY, BRESP, BVALID, ARREADY, RDATA, RRESP, RVALID, RLAST  
);
```

```
modport mem_ram(input ACLK, ARESETn, mem_en, mem_we, mem_addr, mem_wdata, output mem_rdata);  
  
modport mem_ctrl(output mem_en, mem_we, mem_addr, mem_wdata, input mem_rdata, ACLK, ARESETn);
```

```
property P_AWADDR;  
    @(posedge ACLK) disable iff (!ARESETn) $rose(AWVALID) |-> ##[0:$] AWVALID && AWREADY;  
endproperty
```

```
property P_WDATA;  
    @(posedge ACLK) disable iff (!ARESETn) $rose(WVALID) |-> ##[0:$] WVALID && WREADY;  
endproperty
```

```
property P_WRESP;  
    @(posedge ACLK) disable iff (!ARESETn) $rose(BVALID) |-> ##[0:$] BVALID && BREADY;  
endproperty
```

```
property P_ARADDR;  
    @(posedge ACLK) disable iff (!ARESETn) $rose(ARVALID) |-> ##[0:$] ARVALID && ARREADY;  
endproperty
```

```
property P_RDATA;  
    @(posedge ACLK) disable iff (!ARESETn) $rose(RVALID) |-> ##[0:$] RVALID && RREADY;  
endproperty
```

```
AWADDR_handshake :
```

```
assert property (P_AWADDR); // Address write handshake
```

```
WDATA_handshake :
```

```
assert property (P_WDATA); // Write data handshake
```

```
WRESP_handshake :
```

```
assert property (P_WRESP); // Write response handshake
```

```
ARADDR_handshake property P_ARADDR;
```

```
assert property (P_ARADDR); // Address Read handshake
```

```
RDATA_handshake :
```

```
assert property (P_RDATA); // Read data handshake
```

Memory TB:

```
import param_pkg::*;
`include "mem_class.sv"
module mem_TB (
    axi4_if.mem_ctrl mem
);

    mem_class object;
    logic [DATA_SIZE-1:0] golden_memory[logic [MEM_ADDR_SIZE-1:0]];

    task automatic generate_stimulus(mem_class obj);

        assert (obj.randomize())
        else $fatal("Error randomizing");

    endtask

    task automatic drive_stim(mem_class obj);
        mem.mem_en    = obj.mem_en;
        mem.mem_we     = obj.mem_we;
        mem.mem_addr   = obj.mem_addr;
        mem.mem_wdata  = obj.mem_wdata;
    endtask

    task automatic collect_out(mem_class obj);
        obj.mem_rdata = mem.mem_rdata;
    endtask

    task automatic golden_model(mem_class obj);
        if (obj.mem_we && obj.mem_en) golden_memory[obj.mem_addr] = obj.mem_wdata;
    endtask

    task automatic check_write(mem_class obj);
        if (golden_memory.exists(obj.mem_addr)) begin
            if (golden_memory[obj.mem_addr] == obj.mem_wdata)
                $display(
                    "Correct Write: Address is: %h ,Write data : %h, golden_arr write data: %h",
                    obj.mem_addr,
                    obj.mem_wdata,
                    golden_memory[obj.mem_addr]
                );
            end else begin
                $error("Write data : %h and Read data is: %h", obj.mem_wdata, golden_memory[obj.mem_addr]);
            end
        end
    endtask

    task automatic check_read(mem_class obj);
        if (golden_memory.exists(obj.mem_addr)) begin
            if (golden_memory[obj.mem_addr] == obj.mem_rdata) begin

                $display("Correct Read: Address is: %h ,Read data : %h, golden_arr Read data: %h",
                    obj.mem_addr, obj.mem_rdata, golden_memory[obj.mem_addr]);
            end else begin

                $error("Adress: %h, Read data : %h and golden_arr read is: %h", obj.mem_addr,
                    obj.mem_rdata, golden_memory[obj.mem_addr]);
            end
        end else begin
            $display("This address is empty");
        end
    endtask

endtask
```

```

task automatic send_and_check(int size, mem_class obj);
    wait (mem.ARESETn == 1);
    @(negedge mem.ACLK);
    for (int i = 0; i < size; i++) begin

        @(negedge mem.ACLK);
        generate_stimulus(obj);
        drive_stim(obj);
        golden_model(obj);
        @(negedge mem.ACLK);
        collect_out(obj);

        if (obj.mem_we && obj.mem_en) check_write(obj);
        else check_read(obj);

        obj.cg.sample();
        obj.last_addr = obj.mem_addr;
        obj.last_we    = obj.mem_we;
        obj.last_en    = obj.mem_en;
    end
endtask

```

```

        initial begin

            object = new();
            object.integrity_mode = 1;
            send_and_check(50, object);
            $stop;
        end

    endmodule

```

Memory Class

```
import param_pkg::*;

class mem_class;
    rand logic mem_en;
    rand logic mem_we;
    rand logic [MEM_ADDR_SIZE-1:0] mem_addr;
    rand logic [DATA_SIZE-1:0] mem_wdata;
    logic [DATA_SIZE-1:0] mem_rdata;
    logic [MEM_ADDR_SIZE-1:0] last_addr;
    logic integrity_mode = 0;
    logic last_we = 0;
    logic last_en = 0;

    constraint mem_en_c {
        mem_en dist {
            {1'b1} :/ 90,
            {1'b0} :/ 10
        };
    }

    constraint mem_addr_c {
        this.mem_addr dist {
            {10'd0} :/ 5,
            {10'd1023} :/ 5,
            [10'd1 : 10'd1022] :/ 90
        };
    }
}
```



```

function void post_randomize();
    if (this.last_we && this.last_en) begin
        this.mem_addr = this.last_addr;
        this.mem_we    = 0;
        this.mem_en    = 1;
    end

endfunction

/*
constraint mem_we_c {
    | | mem_we == {1'b0};
}
*/

covergroup cg;
    option.auto_bin_max = 0;
    mem_en: coverpoint mem_en {
        bins zero = {1'b0}; bins one = {1'b1}; bins one_to_one = (1'b1 => 1'b1);
    }

    mem_we: coverpoint mem_we {
        bins zero = {1'b0};
        bins one = {1'b1};
        bins one_to_zero = (1'b1 => 1'b0);
        bins zero_to_one = (1'b0 => 1'b1);
    }

```

```

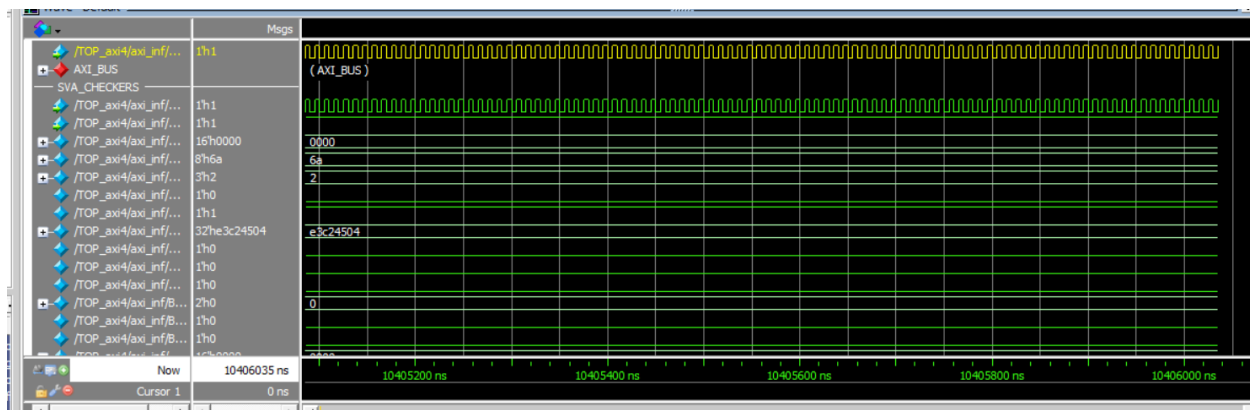
addr: coverpoint mem_addr {
    bins max = {10'd1023}; bins min = {10'd0}; bins mid = {[10'd1 : 10'd1022]};
}

Read_Write: cross mem_en, mem_we{

    ignore_bins ignore = binsof (mem_en.zero);
    bins read = binsof (mem_en.one) && binsof (mem_we.zero);
    bins write = binsof (mem_en.one) && binsof (mem_we.one);
    bins read_to_write = binsof (mem_en.one_to_one) && binsof (mem_we.zero_to_one);
    bins write_to_read = binsof (mem_en.one_to_one) && binsof (mem_we.one_to_zero);
}
endgroup

function new();
    cg = new();
endfunction
endclass

```



Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/TOP_axi4/axi_inf/...	Concurrent	SVA	on	0	2000	0	0B	1608	45 ns	4000	off	assert(@(posedge ACLK) disable if...	✓
/TOP_axi4/axi_inf/...	Concurrent	SVA	on	0	2000	0	0B	1608	55 ns	6000	off	assert(@(posedge ACLK) disable if...	✓
/TOP_axi4/axi_inf/...	Concurrent	SVA	on	0	2000	0	0B	1608	1135 ns	4000	off	assert(@(posedge ACLK) disable if...	✓
/TOP_axi4/axi_inf/...	Concurrent	SVA	on	0	2000	0	0B	1608	1145 ns	4000	off	assert(@(posedge ACLK) disable if...	✓
/TOP_axi4/axi_inf/...	Concurrent	SVA	on	0	257649	0	0B	1608	1185 ns	515298	off	assert(@(posedge ACLK) disable if...	✓
/TOP_axi4/TB/gen...	Immediate	SVA	on	0	2000	0	-	-	-	-	off	assert(randomize(...))	✓

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_insta
/axi4_class/axi4_p...		100.00%					
TYPE cg		100.00%	100	100.00...	✓		
CVP cg::m...		100.00%	100	100.00...	✓		
CVP cg::sla...		100.00%	100	100.00...	✓		
CVP cg::m...		100.00%	100	100.00...	✓		
CVP cg::sla...		100.00%	100	100.00...	✓		
CVP cg::W...		100.00%	100	100.00...	✓		
CVP cg::Bv...		100.00%	100	100.00...	✓		
CVP cg::Br...		100.00%	100	100.00...	✓		
CVP cg::W...		100.00%	100	100.00...	✓		
CVP cg::W...		100.00%	100	100.00...	✓		
CVP cg::W...		100.00%	100	100.00...	✓		
CVP cg::W...		100.00%	100	100.00...	✓		
CVP cg::m...		100.00%	100	100.00...	✓		
CVP cg::sla...		100.00%	100	100.00...	✓		
CVP cg::m...		100.00%	100	100.00...	✓		
CVP cg::sla...		100.00%	100	100.00...	✓		
CVP cg::R_...		100.00%	100	100.00...	✓		
CVP cg::R_...		100.00%	100	100.00...	✓		
CVP cg::R_...		100.00%	100	100.00...	✓		
CVP cg::r_l...		100.00%	100	100.00...	✓		
CVP cg::r_...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
CROSS cg::...		100.00%	100	100.00...	✓		
INST \axi...		100.00%	100	100.00...	✓		

Code Coverage Analysis

Branches - by instance (/TOP_axi4/dut)

Branch

my_axi4.sv

```
42 if (!slave.ARESETn) begin
75 end else begin
84 W_IDLE: begin
89 if (slave.AWVALID && slave.AWREADY) begin
100 W_ADDR: begin
105 W_DATA: begin
106 if (slave.WVALID && slave.WREADY) begin
107 if (write_addr_valid && !write_boundary_cross) begin
114 if (slave.WLAST || write_burst_cnt == 0) begin
118 if (!write_addr_valid || write_boundary_cross) begin
120 end else begin
124 end else begin
131 W_RESP: begin
132 if (slave.BREADY && slave.BVALID) begin
140 default: write_state <= W_IDLE;
147 R_IDLE: begin
152 if (slave.ARVALID && slave.ARREADY) begin
163 R_ADDR: begin
164 if (read_addr_valid && !read_boundary_cross) begin
171 R_WAIT: begin // new added state
175 R_DATA: begin
176 if (read_addr_valid && !read_boundary_cross) begin
179 end else begin
187 if (slave.RREADY && slave.RVALID) begin
190 if (read_burst_cnt > 0) begin
194 if (read_addr_valid && !read_boundary_cross) begin
202 end else begin
209 default: read_state <= R_IDLE;
```

Code Coverage Analysis

Statements - by instance (/TOP_axi4/dut)

Statement

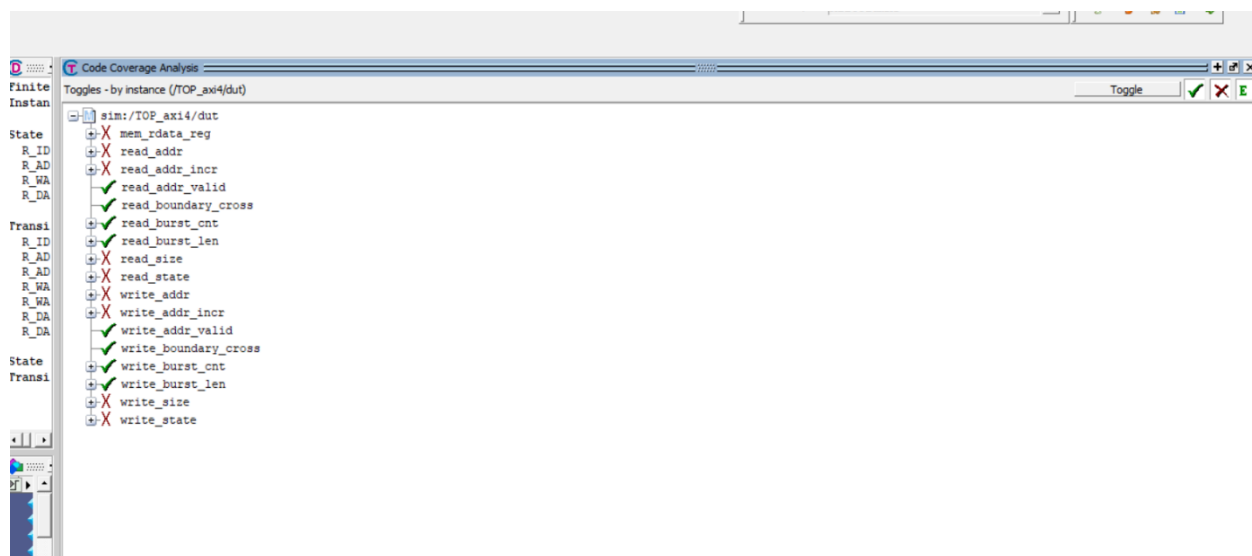
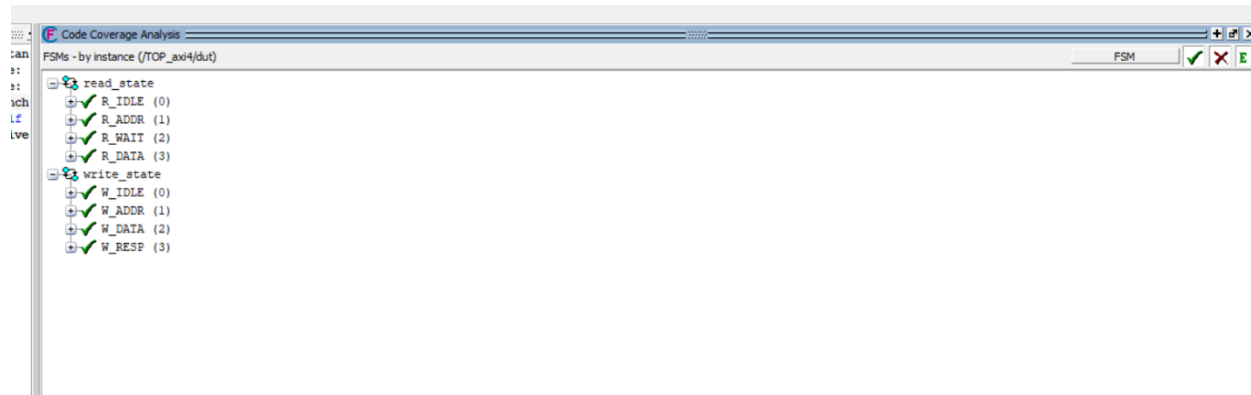
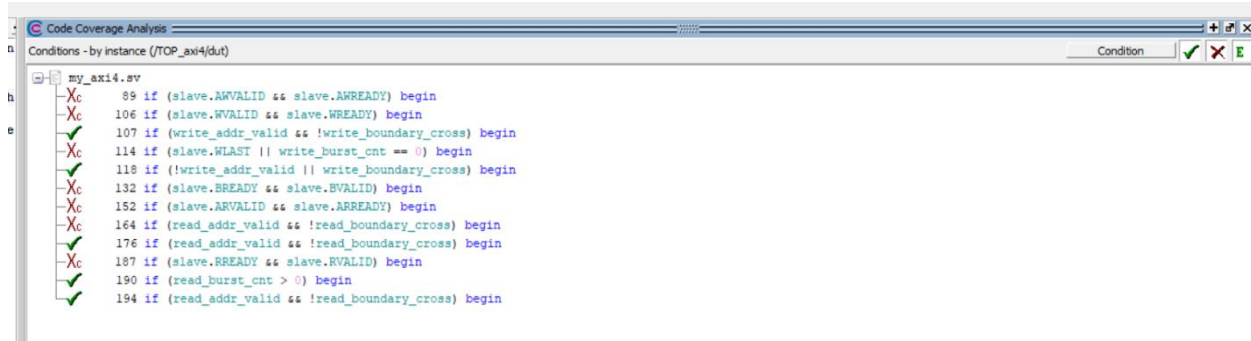
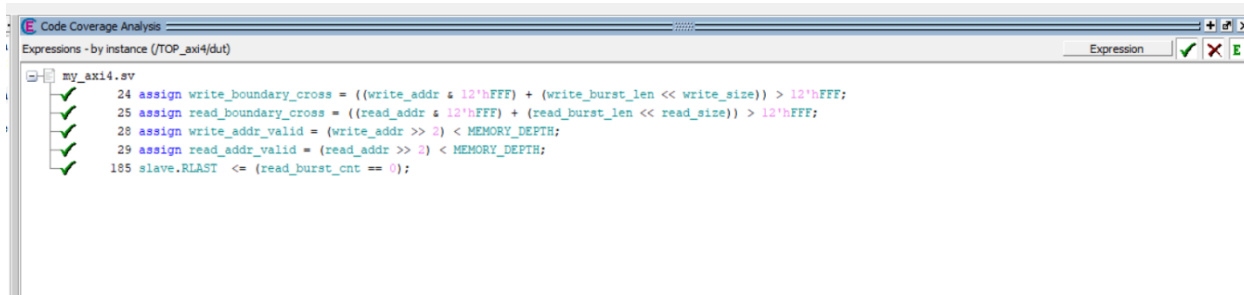
my_axi4.sv

```
20 assign write_addr_incr = (1 << write_size);
21 assign read_addr_incr = (1 << read_size);
24 assign write_boundary_cross = ((write_addr & 12'hFFF) + (write_burst_len << write_size)) > 12'hFFF;
25 assign read_boundary_cross = ((read_addr & 12'hFFF) + (read_burst_len << read_size)) > 12'hFFF;
28 assign write_addr_valid = (write_addr >> 2) < MEMORY_DEPTH;
29 assign read_addr_valid = (read_addr >> 2) < MEMORY_DEPTH;
41 always @(posedge slave.ACLK or negedge slave.ARESETn) begin
44 slave.AWREADY <= 1'b1;
45 slave.WREADY <= 1'b0;
46 slave.BVALID <= 1'b0;
47 slave.BRESP <= 2'b00;
49 slave.ARREADY <= 1'b1;
50 slave.RVALID <= 1'b0;
51 slave.RRESP <= 2'b00;
52 slave.RDATA <= (DATA_WIDTH{1'b0});
53 slave.RLAST <= 1'b0;
56 write_state <= W_IDLE;
57 read_state <= R_IDLE;
58 memory.mem_en <= 1'b0;
59 memory.mem_we <= 1'b0;
60 memory.mem_addr <= ($clog2(MEMORY_DEPTH) {1'b0});
61 memory.mem_wdata <= (DATA_WIDTH{1'b0});
64 write_addr <= (ADDR_WIDTH{1'b0});
65 read_addr <= (ADDR_WIDTH{1'b0});
66 write_burst_len <= 8'b0;
67 read_burst_len <= 8'b0;
68 write_burst_cnt <= 8'b0;
69 read_burst_cnt <= 8'b0;
```

Statements - by instance (/TOP_axi4/dut)

Statement

```
70 write_size <= 3'b0;
71 read_size <= 3'b0;
73 mem_rdata_reg <= (DATA_WIDTH{1'b0});
77 memory.mem_en <= 1'b0;
78 memory.mem_we <= 1'b0;
85 slave.AWREADY <= 1'b1;
86 slave.WREADY <= 1'b0;
87 slave.BVALID <= 1'b0;
90 write_addr <= slave.AWADDR;
91 write_burst_len <= slave.AWLEN;
92 write_burst_cnt <= slave.AWLEN;
93 write_size <= slave.AWSIZE;
95 slave.AWREADY <= 1'b0;
96 write_state <= W_ADDR;
101 slave.WREADY <= 1'b1;
102 write_state <= W_DATA;
108 memory.mem_en <= 1'b1;
109 memory.mem_we <= 1'b1;
110 memory.mem_addr <= write_addr >> 2;
111 memory.mem_wdata <= slave.WDATA;
115 slave.WREADY <= 1'b0;
116 write_state <= W_RESP;
119 slave.BRESP <= 2'b10; // SLVERR
121 slave.BRESP <= 2'b00; // OKAY
123 slave.BVALID <= 1'b1;
125 write_addr <= write_addr + write_addr_incr;
126 write_burst_cnt <= write_burst_cnt - 1'b1;
133 slave.BVALID <= 1'b0;
134 slave.BRESP <= 2'b00;
136 write_state <= W_IDLE;
```



= axi4_interface.sv 3 X = my_axi4.sv = TOP_axi4.sv 1 X = axi4_class.sv 34

☰ TOP_axi4.sv > ...

```
1  module TOP_axi4;
2      bit clk = 0;
3      bit rst_n;
4
5      initial begin
6          forever begin
7              #5ns clk = ~clk;
8          end
9      end
10
11     initial begin
12         rst_n = 0;
13         #20ns rst_n = 1;
14     end
15
16     axi4_if axi_inf (
17         .CLK(clk),
18         .ARESETn(rst_n)
19     );
20     my_TB TB (axi_inf.master);
21     my_axi4_memory memory (axi_inf.mem_ram);
22     my_axi4 dut (axi_inf.slave, axi_inf.mem_ctrl);
23
24     endmodule
25
```

```

7 # 0: Runs in batch mode for regressions.
8 set DEBUG_MODE 1
9
10 # 1. Cleanup and Library Setup
11 if {[file exists work]} { vdel -all }
12 vlib work
13
14 # 2. Compile Phase
15 vlog param_pkg.sv axi4_interface.sv axi4_class.sv my_TB.sv TOP_axi4.sv
16 vlog +cover=sbcetf my_axi4.sv my_axi4_memory.sv
17
18 # 3. Execution Branch
19 if {$DEBUG_MODE == 1} {
20     puts "--- Running in DEBUG MODE (GUI + Waves + Coverage + SVA) ---"
21
22     # -assertcover forces SVA data into the database
23     vsim -coverage -assertdebug -assertcover -voptargs="+acc +cover=sbcetf" work.TOP_axi4 -do {
24
25         # 1. Add Waves
26         add wave -color Yellow -itemcolor Yellow /TOP_axi4/axi_inf/ACLK
27         add wave -group "AXI_BUS" /TOP_axi4/axi_inf/*
28         add wave -divider "SVA_CHECKERS"
29         add wave -assert /TOP_axi4/axi_inf/*
30         add wave -divider "TB_MONITOR"
31         add wave -radix decimal /TOP_axi4/TB/operation
32         add wave /TOP_axi4/TB/flag
33         add wave -divider "DUT_INTERNAL"
34         add wave /TOP_axi4/dut/*
35
36         # 2. Open Assertion Tracking Window
37         view assertions
38     }
39 }

```

```

vsim -coverage -assertdebug -assertcover -voptargs="+acc +cover=sbcetf" work.TOP_axi4 -do {
    # 3. Load saved wave format if it exists
    if {[file exists wave.do]} {
        puts "Loading saved wave format..."
        do wave.do
    }

    # 4. Save coverage database when simulation ends
    coverage save -onexit cov.ucdb;

    # 5. Run the simulation
    run -all;

    # 6. Auto-generate the Master Text Report (Code + CVG + Assertions)
    coverage report -detail -cvg -assert -file full_coverage_report.txt
    puts "--- Text Coverage Report Generated: full_coverage_report.txt ---"
}

else {
    puts "--- Running in REGRESSION MODE ---"

    set seed [expr {int(rand() * 1000000)}]
    vsim -c -voptargs="+acc +cover=sbcetf" -coverage -assertcover work.TOP_axi4 -sv_seed $seed -do "
        coverage save -onexit cov.ucdb;
        run -all;
        quit -sim;
    "
}

```

```

# Generate the Master Text Report in batch mode
vcover report cov.ucdb -detail -cvg -assert -file full_coverage_report.txt
puts "--- Regression Complete. Report Generated: full_coverage_report.txt ---"

```

